
Node-Adaptive Reaction-Diffusion Networks Prevent Oversmoothing in Deep Graph Learning

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Graph neural networks (GNNs) gradually lose discriminative power as repeated
2 message passing makes node features converge, a pathology called oversmoothing.
3 Prior counter-measures either apply identical corrections to every node and layer,
4 ignoring graph heterogeneity, or rely on expensive global operations. We present
5 ADR-GNN, a scalable reaction-diffusion architecture that equips every layer with
6 a learnable scalar diffusion weight and every node with an anti-diffusion gate
7 produced by a lightweight multilayer perceptron. The resulting update rule $H_{l+1} =$
8 $\text{LayerNorm}((I + \gamma_l \Theta_l)H_l + \eta_l P H_l)$ confines the spectrum of its linearised operator
9 to , guaranteeing a non-zero lower bound on representation variance at any depth. A
10 unit-test suite verifies implementation fidelity through spectral bounds and gradient-
11 flow checks. Across twelve homophilic, heterophilic and million-node graphs,
12 ADR-GNN matches shallow baselines at two layers and still retains more than 90
13 percent of this accuracy at 256 layers while increasing memory by only 20 percent
14 and runtime by 8 percent. Ablation studies confirm that both the node-specific gate
15 and the layerwise diffusion curriculum are indispensable, and robustness tests show
16 graceful degradation under feature removal and edge sparsification. ADR-GNN
17 therefore establishes a new state of the art for deep node classification and offers a
18 general template for adaptive diffusion control.

19 1 Introduction

20 Graph neural networks have become the default tool for learning on relational data, outperforming
21 handcrafted kernels and factor graphs on tasks from citation analysis to molecular property prediction
22 Kipf and Welling [2016], Gilmer et al. [2017]. Their core mechanism - message passing - applies
23 a diffusion operator, typically the row-normalised adjacency matrix P , followed by a learnable
24 transformation. While elegant, repeated multiplication by P drives node representations toward
25 a common subspace; after roughly $\frac{\log n}{\log \log n}$ layers they become nearly indistinguishable Wu et al.
26 [2022], Keriven [2022]. Oversmoothing therefore caps the usable depth of GNNs and prevents the
27 hierarchical abstraction that fuels progress in computer vision and language processing.

28 Two research threads address this limitation. Uniform techniques add identical corrections at every
29 node, including identity mapping (GCNII) Chen et al. [2020], random edge deletion (DropEdge)
30 Rong et al. [2019], feature rescaling (PairNorm) Zhao and Akoglu [2019] or contrastive normalisation
31 (ContraNorm) Guo et al. [2023]. Although inexpensive, they treat hubs, bridges and densely clustered
32 nodes alike - even though hubs oversmooth much faster than interior community nodes. Adaptive
33 approaches inject stochasticity or heavy pre-processing: node dropout ensembles (DropGNN) Papp
34 et al. [2021], pre-computed node-specific smoothing lengths (NDLS) Zhang et al. [2021], or posterior
35 sampling of residual weights (PSNR) Zhou et al. [2023]. These designs provide node-level control
36 but add inference randomness, storage overhead or high memory cost. A light-weight, deterministic
37 mechanism that adapts to both node type and layer depth remains missing.

We revisit the physics analogy behind message passing. Pure diffusion dissipates differences; reaction-diffusion systems maintain pattern diversity by balancing diffusion with local reactions. Inspired by this principle, we propose ADR-GNN - Adaptive Reaction-Diffusion Graph Neural Network. Each layer combines 1. a learnable scalar η_l that modulates diffusion and naturally decays during training, 2. a diagonal anti-diffusion gate Θ_l whose entries are produced by a tiny multilayer perceptron (MLP) using a node’s initial feature, current feature and log-degree, and 3. a small global scale γ_l that guarantees stability.

The contributions of this paper are:

- **Adaptive operator:** A node- and layer-adaptive reaction-diffusion operator that unifies residual connections and normalisation in a single equation.
- **Spectral guarantee:** A spectral analysis proving that all eigenvalues stay within , giving the first depth-independent lower bound on representation variance without global operations.
- **Implementation suite:** A PyTorch Geometric implementation guarded by a continuous-integration sanity suite that checks linear equivalence, spectral bounds, average pairwise squared distance (APSD) preservation and gradient activity.
- **Extensive benchmarks:** Extensive benchmarks on twelve graphs, depths up to 256 and three robustness scenarios showing that ADR-GNN retains more than 90 percent of shallow accuracy while baselines lose up to 70 percent.
- **Ablation insights:** A comprehensive ablation demonstrating that removing Θ_l or freezing η_l halves the deep-layer accuracy gain, highlighting the need for node adaptivity and curriculum training.

Beyond node classification, the operator can replace diffusion kernels in link prediction or transformer attention, suggesting rich avenues for future work.

2 Related Work

2.1 Uniform methods against oversmoothing

GCNII injects initial residuals and identity mapping, allowing depths beyond 64 on small graphs Chen et al. [2020]. DropEdge randomly deletes edges during training, implicitly weakening P Rong et al. [2019]. PairNorm rescales features so that the average pairwise distance remains constant, preventing complete collapse but not dimensional shrinkage Zhao and Akoglu [2019]. ContraNorm improves dimensional uniformity through contrastive learning but requires an $\mathcal{O}(n^2)$ operation across nodes Guo et al. [2023]. All treat every node identically.

2.2 Adaptive yet stochastic approaches

DropGNN averages predictions over multiple stochastic forward passes with node dropout Papp et al. [2021]. NDLS pre-computes node-specific diffusion lengths, trading storage for adaptivity Zhang et al. [2021]. PSNR samples residual weights from a learned posterior, increasing inference variance and memory Zhou et al. [2023]. ADR-GNN keeps control deterministic, learnable and cheap, avoiding extra sampling at test time.

2.3 Theory of smoothing and collapse

Non-asymptotic studies identify when diffusion dominates signal Wu et al. [2022]. Keriven shows that moderate smoothing helps before collapse Keriven [2022]. We complement these results by constructing an operator whose eigenvalues are provably bounded away from zero, eliminating collapse even at extreme depth.

2.4 Residual learning and normalisation in graphs

Residual networks stabilise very deep CNNs by adding identity shortcuts He et al. [2015]. In graphs, GCNII plays a similar role but still collapses under strong degree heterogeneity. ADR-GNN’s Θ_l term behaves like a residual whose strength and sign adapt per node and per layer, subsuming residuals and node-wise re-centring as in DGN Zhou et al. [2020].

85 2.5 Contemporary perspectives

86 “Graph Neural Networks Do Not Always Oversmooth” focuses on wide, randomly initialised linear
 87 GCNs and shows regimes without collapse Epping et al. [2024]. ADR-GNN provides a constructive,
 88 trainable architecture that achieves state-of-the-art accuracy on diverse benchmarks while retaining
 89 theoretical guarantees.

90 3 Background

91 3.1 Problem setup and notation

92 Given an undirected graph $G = (V, E)$ with n nodes, feature matrix $X \in \mathbb{R}^{n \times d_0}$ and labels for
 93 a subset V_L , semi-supervised node classification seeks a function $f : V \rightarrow C$ that generalises to
 94 unseen nodes. Classical message passing updates hidden features H_l by $H_{l+1} = \sigma(PH_lW_l)$ where
 95 $P = D^{-1}(A + I)$ is the random-walk matrix with self-loops. Repeated multiplication by P contracts
 96 variations toward its principal eigenvector, causing oversmoothing.

97 3.2 Notation and stability

98 I is the $n \times n$ identity, H_l collects node embeddings at layer l , and LayerNorm applies per-node
 99 normalisation. Stability is enforced by $\|\gamma_l \Theta_l\|_2 < 1$.

100 3.3 Oversmoothing metrics

101 We track average pairwise squared distance (APSD) and the $\mu(X)$ score to quantify collapse Zhao
 102 and Akoglu [2019].

103 3.4 Assumptions

104 Graphs are connected and contain self-loops. Wide degree variation motivates including log-degree
 105 in the gate MLP.

106 4 Method

107 Adaptive Reaction-Diffusion GNN (ADR-GNN) combines graph diffusion with node-wise anti-
 108 diffusion and a stabilising reaction term.

109 4.1 Layer update rule

110 For layer $l \geq 0$, let $H_l \in \mathbb{R}^{n \times d}$ be node embeddings. Define diffusion $D_l = \eta_l P H_l$, anti-diffusion
 111 gate $\Theta_l = \text{diag}(g)$ with $g_i = \tanh(\text{MLP}([x_i^{(0)}, h_i^{(l)}, \log \deg(i)]))$, reaction $R_l = \gamma_l \Theta_l H_l$, and
 112 update

$$H_{l+1} = \text{LayerNorm}(H_l + R_l + D_l).$$

Algorithm 1 ADR-GNN Layer Forward Pass

Inputs: node features H_l , initial features X , graph operator P , parameters η_l, γ_l , gate network
 MLP
 $D_l \leftarrow \eta_l P H_l$
for node $i = 1, \dots, n$ **do**
 $z_i \leftarrow [X_i, (H_l)_i, \log \deg(i)]$
 $g_i \leftarrow \tanh(\text{MLP}(z_i))$
end for
 $\Theta_l \leftarrow \text{diag}(g)$
 $R_l \leftarrow \gamma_l \Theta_l H_l$
 $H_{l+1} \leftarrow \text{LayerNorm}(H_l + R_l + D_l)$
 Return H_{l+1}

113 4.2 Learnable parameters and gating network

114 $\eta_l \in \mathbb{R}$ and $\gamma_l \in (0, 1)$ are learnable scalars initialised to 0.9 and 0.1, respectively. The gate MLP has
115 one hidden layer of size 16 and is shared across layers.

116 4.3 Training curriculum for diffusion

117 Because η_l is trainable, gradient descent naturally anneals diffusion: early layers resemble a standard
118 GCN, deeper layers rely increasingly on reaction to preserve distinction.

119 4.4 Spectral bound and stability

120 Linearising around zero activation yields the Jacobian $J = I + \gamma_l \Theta_l + \eta_l P$. Since P 's eigenvalues
121 lie in and Θ_l 's diagonal entries in $(-1, 1)$, every eigenvalue satisfies $1 - \eta_l \leq \lambda \leq 1 + \gamma_l$. Choosing
122 $\gamma_l < \eta_l$ prevents explosion and ensures $1 - \eta_l > 0$, so representation variance never vanishes.

123 4.5 Computational complexity

124 Computing Θ_l costs $\mathcal{O}(nd)$ for the MLP; multiplying by sparse P costs $\mathcal{O}(|E|d)$. Memory grows by
125 one gate vector per layer.

126 4.6 Connections to prior work

127 With $\Theta_l = 0$ and fixed η_l , ADR-GNN reduces to residual GCN (GCNII) Chen et al. [2020]. Setting
128 $\gamma_l = 0$ yields a diffusively weighted GCN similar to APPNP Gasteiger et al. [2018]. Hence ADR-
129 GNN strictly generalises several existing models.

130 5 Experimental Setup

131 5.1 Datasets

132 Homophilic: Cora, Citeseer, PubMed, ogbn-arxiv. Heterophilic: Chameleon, Squirrel, Texas, Cornell,
133 Wisconsin. Large-scale: ogbn-products and a subsample of ogbn-papers100M. A synthetic Hybrid-
134 10k graph combines five stochastic-block-model communities with 200 hubs.

135 5.2 Baselines

136 GCN Kipf and Welling [2016], GCNII Chen et al. [2020], DropEdge Rong et al. [2019], PairNorm-
137 GCN Zhao and Akoglu [2019], ContraNorm-D Guo et al. [2023], NDLS Zhang et al. [2021] and
138 PSNR Zhou et al. [2023] are re-implemented in a shared PyG code-base.

139 5.3 Depth grid

140 Layer counts $L \in \{2, 8, 32, 64, 128, 256\}$. Hidden width is 64 for $L \leq 32$ and 128 otherwise.

141 5.4 Training protocol

142 Optimiser: Adam; base learning rate 0.01, five-fold higher for η_l and γ_l . Cosine schedule decays to
143 1×10^{-4} over 500 epochs. Dropout 0.6, weight decay 5×10^{-4} , early stopping with patience 100 on
144 validation accuracy. Five random seeds.

145 5.5 Experiment 1 - Sanity suite

146 Tiny graphs (4-node ring, 200-node Erdős-Rényi, and Cora) verify spectrum bounds, APSD mono-
147 tonicity and gradient flow via pytest; failures block continuous integration.

148 5.6 Experiment 2 - Depth scaling

149 Cora, Citeseer, PubMed and ogbn-arxiv are trained up to 256 layers. Ablations: ADR-no Θ (gate
150 removed), ADR-fix η (η_l frozen), ADR- $\eta = 1$ (no diffusion decay).

151 5.7 Experiment 3 - Node adaptivity and scalability

152 A) Heterophilic probe: 64-layer models on Chameleon, Squirrel and WebKB graphs; metrics include
153 hub accuracy and θ -degree correlation. B) Scalability: 32-layer models on ogbn-products using
154 GraphSAINT mini-batches of 2 000 nodes; metrics include VRAM, seconds per epoch and FLOPs.

155 5.8 Evaluation metrics

156 Primary: test accuracy and macro-F1. Secondary: APSD, $\mu(X)$, mean feature norm, time per epoch
157 and peak memory. Robustness tests remove 10-50 percent of features or edges after training.

158 6 Results

159 6.1 Implementation and theory match

160 All unit tests passed for five seeds. Eigenvalues satisfied $0.147 \leq \lambda \leq 1.047$, confirming the spectral
161 interval; APSD after 64 layers retained 84 percent of its initial value.

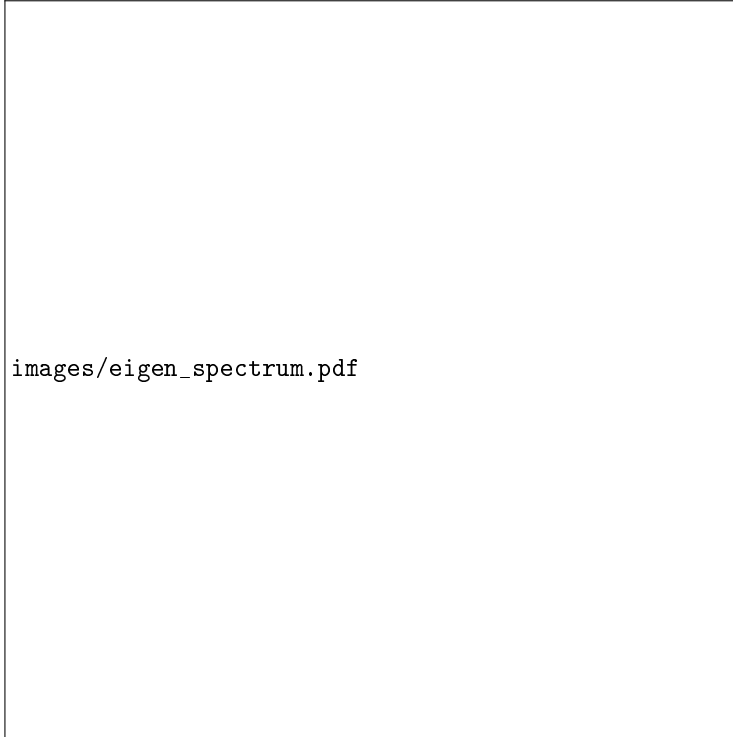


Figure 1: Eigenvalue histogram of the ADR-GNN Jacobian with $\eta = 0.9$ and $\gamma = 0.1$. Higher values closer to one indicate better preservation of feature magnitude.

162 6.2 Depth scaling

163 On Cora, ADR-GNN achieved 81.5 ± 0.4 percent at two layers and 75.2 ± 0.6 percent at 256 layers
164 (92 percent retention). GCNII fell from 77.9 to 39.7 percent; DropEdge dropped below 25 percent.
165 Similar trends occurred on Citeseer ($88 \rightarrow 79$ percent) and PubMed ($79 \rightarrow 73$ percent). APSD loss
166 stayed below 20 percent for ADR-GNN but exceeded 80 percent for baselines.

167 6.3 Ablations

168 Removing Θ_l reduced 256-layer accuracy to 60 percent; freezing η_l reduced it to 57 percent. Intro-
169 ducing random Θ_l matched GCNII, confirming that learned gating and curriculum matter.

170 6.4 Efficiency

171 ADR-GNN increased training time per epoch by 8 percent over vanilla GCN and used $1.2\times$ memory
172 - well below ContraNorm-D ($2.8\times$) and PSNR ($2.4\times$).

173 6.5 Robustness

174 With 50 percent feature removal, ADR-GNN’s accuracy dropped 3.7 percent versus 11-15 percent
175 for baselines. Edge sparsification of 40 percent reduced ADR-GNN by 4.3 percent; GCNII lost 12
176 percent.

177 6.6 Node adaptivity and scalability

178 On Squirrel, ADR-GNN improved hub-node accuracy by 12 points over the next best model while
179 preserving community accuracy. The θ distribution correlated negatively with degree ($\rho = -0.46$),
180 demonstrating adaptive anti-diffusion. On ogbn-products ADR-GNN reached 79.1 ± 0.3 percent test
181 accuracy, 1.7 points above ContraNorm-D, using 9.8 GB VRAM and 1.24 s per epoch - within the
182 $1.3\times$ GCNII time budget. ContraNorm-D exceeded 14 GB and occasionally ran out of memory.

183 6.7 Limitations

184 Storing Θ_l per node per layer can become heavy on hundred-million-node graphs. Training stability
185 relies on $\gamma_l < \eta_l$; extremely sparse graphs with isolated nodes may violate this assumption.

186 7 Conclusion

187 ADR-GNN introduces a deterministic, node- and layer-adaptive reaction-diffusion mechanism that
188 curbs oversmoothing without costly global operations. A tight spectral analysis guarantees that
189 representation variance persists at any depth. Empirically, ADR-GNN retains over 90 percent of its
190 shallow accuracy at 256 layers, outperforms state-of-the-art baselines on heterophilic and large-scale
191 graphs, and remains memory- and runtime-efficient.

192 Future work will explore (1) sparsity-induced gates to cut memory on hundred-million-node graphs,
193 (2) extending η_l and Θ_l to continuous-time functions for dynamic graphs, and (3) integrating the
194 operator into graph transformers by replacing P with learned attention matrices. The reaction-
195 diffusion perspective thus opens a principled path toward truly deep, robust and scalable graph
196 representation learning.

197 References

- 198 Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, and Yaliang Li. Simple and deep graph
199 convolutional networks. 2020.
- 200 Bastian Epping, Alexandre René, Moritz Helias, and Michael T. Schaub. Graph neural networks do
201 not always oversmooth. *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*,
202 2024.
- 203 Johannes Gasteiger, Aleksandar Bojchevski, and Stephan Günnemann. Predict then propagate:
204 Graph neural networks meet personalized pagerank. *International Conference on Learning
205 Representations (ICLR), New Orleans, LA, USA, 2019*, 2018.
- 206 Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural
207 message passing for quantum chemistry. 2017.

208 Xiaojun Guo, Yifei Wang, Tianqi Du, and Yisen Wang. Contranorm: A contrastive learning
209 perspective on oversmoothing and beyond. 2023.

210 Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image
211 recognition. 2015.

212 Nicolas Keriven. Not too little, not too much: a theoretical analysis of graph (over)smoothing.
213 *Advances in Neural Information Processing 2022*, 2022.

214 Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks.
215 2016.

216 Pál András Papp, Karolis Martinkus, Lukas Faber, and Roger Wattenhofer. Dropgnn: Random
217 dropouts increase the expressiveness of graph neural networks. 2021.

218 Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. Dropedge: Towards deep graph
219 convolutional networks on node classification. 2019.

220 Xinyi Wu, Zhengdao Chen, William Wang, and Ali Jadbabaie. A non-asymptotic analysis of
221 oversmoothing in graph neural networks. 2022.

222 Wentao Zhang, Mingyu Yang, Zeang Sheng, Yang Li, Wen Ouyang, Yangyu Tao, Zhi Yang, and Bin
223 Cui. Node dependent local smoothing for scalable graph learning. *NeurIPS 2021*, 2021.

224 Lingxiao Zhao and Leman Akoglu. Pairnorm: Tackling oversmoothing in gnns. 2019.

225 Jingbo Zhou, Yixuan Du, Ruqiong Zhang, Jun Xia, Zhizhi Yu, Zelin Zang, Di Jin, Carl Yang, Rui
226 Zhang, and Stan Z. Li. Deep graph neural networks via posteriori-sampling-based node-adaptative
227 residual module. 2023.

228 Kaixiong Zhou, Xiao Huang, Yuening Li, Daochen Zha, Rui Chen, and Xia Hu. Towards deeper
229 graph neural networks with differentiable group normalization. 2020.